

AGORA

Platform Specification Book

Medical Education Platform — Full Technical & Product Specification

Version 1.0 | 2026

Frontend: Next.js • Backend: Supabase / Django (dual spec)

CONFIDENTIAL — INTERNAL DOCUMENT

1. PROJECT OVERVIEW

1.1 Introduction

Agora is an educational platform dedicated to medical students, designed to help them navigate their studies through structured lessons, interactive quizzes, and competitive gamification. It combines academic rigor with modern engagement mechanics to keep students motivated, consistent, and progressing.

1.2 Mission Statement

"To create a structured, engaging, and competitive learning ecosystem where medical students can study, practise, compete, and grow continuously."

1.3 Platform Name & Identity

Attribute	Value
Project Name	Agora
Tagline	Learn. Compete. Excel.
Category	EdTech — Medical Education
Visual Style	Dark academia — Green / Beige / Blue
Primary Language	French (expandable to multilingual)
Target Platform	Web (Desktop-first, Mobile-responsive)

1.4 Goals & Objectives

- Provide structured, simplified, and interactive medical content
- Gamify the study process to improve retention and motivation
- Foster a competitive and social learning environment
- Track individual performance over time
- Build a scalable platform ready for premium monetisation

2. TARGET USERS & USER PERSONAS

■ 2.1 Primary Audience

Medical students at any year of study who need structured practice, interactive content, and competitive engagement to stay motivated throughout long programmes.

■ 2.2 User Personas

▶ Persona A — The Grinder

- 2nd-year medical student, studying 8–10 h/day
- Needs quick MCQ practice between lectures
- Cares about streaks and leaderboard rankings
- Uses Blitz Mode and Free Mode daily

▶ Persona B — The Social Learner

- 3rd-year student, studies in groups
- Loves collaborative quiz sessions
- Uses Room Mode with friends
- Values chat and real-time competition

▶ Persona C — The Analyst

- Focused on identifying weak subjects
- Studies statistics dashboards carefully
- Plans study sessions based on performance gaps
- Uses all modes but driven by metrics

■ 2.3 Future Expansion

- Pharmacy students
- Dentistry students
- Paramedical studies (nursing, physiotherapy)
- Science undergraduate programmes

3. PLATFORM ARCHITECTURE & SECTIONS

3.1 Four Main Sections

Agora is structured into four core sections, each serving a distinct purpose:

Section Map	
1. LESSONS	— Structured educational content
2. MEDQUEST	— Interactive quiz system (3 modes)
3. STATISTICS	— Performance tracking & analytics
4. BLOG	— Editorial content & platform updates

3.2 Lessons

► Overview

The Lessons section is the academic backbone of Agora. All content is rewritten, simplified, and structured for clarity and quick understanding — unlike raw textbooks.

► Content Structure

Level	Description
Subject	Top-level grouping (e.g. Cardiology, Anatomy, Biochemistry)
Unit	Sub-grouping within a subject (e.g. Cardiac Physiology)
Module	Individual lesson (e.g. The Cardiac Cycle)
Section	Sub-part of a module (e.g. Systole vs Diastole)

► Content Format

- Rewritten in simplified, exam-focused language
- Illustrated with diagrams (images or SVG-based visuals)
- Demo-based interactive explanations where applicable
- Summary boxes at the end of each module
- Linked to relevant QCMs in MedQuest

► Admin Functionality

- Admins can create / edit / delete subjects, units, modules
- Supports rich text editor (Markdown or WYSIWYG)
- Image upload and embedding
- Module visibility toggle (draft / published)

■ 3.3 MedQuest — Quiz System

► Overview

MedQuest is the competitive engine of Agora. It offers three distinct quiz modes catering to solo practice, speed challenges, and social competition.

► Mode 1 — Free Mode

Free Mode
Purpose : Solo self-assessment practice
Format : QCM (Multiple Choice Questions)
Pacing : No time pressure — user proceeds at own speed
Selection : User selects subject + lesson
Scoring : Correct / Incorrect shown after each answer
Review : Full explanation shown per question
History : Session results saved to Statistics

► Mode 2 — Blitz Mode

Blitz Mode
Purpose : Speed & accuracy challenge
Format : QCM timed sequence
Timer : Countdown starts at e.g. 60 seconds
Correct + : +5 seconds added
Wrong - : -5 seconds removed
End State : Timer hits 0 → session ends
Score : Total correct answers + longest streak
Leaderboard: Global and friend leaderboards per lesson

► Mode 3 — Room Mode

Room Mode
Purpose : Multiplayer real-time quiz competition
Access : Private room (invite code)
Setup : Host selects subject + lesson + question count
Capacity : 2–10 players per room
Chat : In-room live chat system
Timer : Per-question countdown (e.g. 20 seconds each)
Scoring : Points based on correct answers + speed

Result : Final leaderboard at session end

History : Match saved in Statistics for all players

► QCM Data Model

Field	Description
id	Unique question identifier (UUID)
subject_id	Associated subject
lesson_id	Associated lesson/module
question_text	The question (plain text or Markdown)
options	Array of 4 choices (A, B, C, D)
correct_option	Index of correct answer (0-3)
explanation	Explanation shown after answer
difficulty	easy / medium / hard
created_at	Timestamp
updated_at	Last edit timestamp

■ 3.4 Statistics

► Tracked Metrics

- Total questions answered
- Total correct / incorrect answers
- Accuracy rate per subject / lesson
- Session history (date, mode, score)
- Streak counter (daily activity)
- Blitz Mode: best scores, best streak
- Room Mode: win/loss record, match history
- Progress chart (accuracy over time)
- Weak subjects & recommendations

► Dashboard Design

- Visual charts: line charts, radar charts, bar charts
- Filter by time range (week / month / all time)
- Comparison against platform average (future)
- Exportable reports (PDF — future premium feature)

■ 3.5 Blog

The Blog section is currently a placeholder. Future content will include:

- Study guides and tips written by contributors

- Platform updates and changelogs
- Medical news curated for students
- Partner content (future)

Technical implementation: A simple CMS-backed section. Posts stored as Markdown or rich-text in the database. Admin-only publishing. Reader comments in future phases.

4. USER SYSTEM & AUTHENTICATION

4.1 Authentication Flows

Flow	Details
Sign Up	Email + password or OAuth (Google)
Login	Email/password or OAuth, JWT session
Password Reset	Email magic link / token reset
Email Verification	Required before full access
Session Management	JWT access + refresh tokens, auto-renew
Logout	Token invalidation

4.2 User Profile Data Model

Field	Type / Description
id	UUID — Primary key
username	String, unique, public display name
email	String, unique, private
password_hash	Hashed (bcrypt/argon2)
avatar_url	String (optional image URL)
year_of_study	Integer (1-7)
university	String (optional)
streak_current	Integer — current daily streak count
streak_best	Integer — all-time best streak
last_activity_date	Date — for streak calculation
role	Enum: student / admin
is_premium	Boolean (for future paid tier)
created_at	Timestamp
updated_at	Timestamp

4.3 Friend System

► Features

- Send / accept / reject friend requests
- View friend list with online status
- Search users by username

- Remove friends
- Challenge a friend to Blitz Mode (future)
- Invite friends directly to a Room

► Friend Data Model

Field	Description
id	UUID
requester_id	User who sent the request
recipient_id	User who received the request
status	pending / accepted / rejected / blocked
created_at	Timestamp

■ 4.4 Daily Streak System

A streak is maintained by completing at least one activity per day (any quiz session or lesson reading). The system resets at midnight local time or UTC.

- Streak increments if activity occurred today and yesterday was also active
- Streak resets to 0 if a day is missed
- Best streak is preserved even if current resets
- Future: streak freeze (premium) allows skipping one day

5. GAMIFICATION & SOCIAL FEATURES

■ 5.1 Core Engagement Loops

Agora uses multiple engagement loops to keep users returning daily:

Loop	Mechanism
Daily Habit	Streak system resets daily — powerful loss-aversion trigger
Mastery	Accuracy improves over repeated attempts per lesson
Competition	Blitz leaderboard + Room Mode vs friends
Social Proof	Friend feed shows activity (future)
Progress	Statistics dashboard shows visual growth
Achievement	Badges for milestones (future phase)

■ 5.2 Leaderboards

- Blitz Mode: global + friends-only leaderboard per lesson
- Room Mode: real-time score display + end-of-game result
- Weekly reset option (configurable per board)
- Top 10 displayed publicly, user's own rank always shown

■ 5.3 Notification System (Future)

- Friend request notifications
- Room invite notifications
- Streak reminder (daily push / email)
- New lesson published notifications
- Weekly performance summary email

■ 5.4 Future Features (Roadmap)

- Achievement badges system
- XP points and levelling
- Global ranked seasons
- Public profile pages
- Study group rooms (persistent)

6. DESIGN SYSTEM & UI GUIDELINES

6.1 Visual Identity

Attribute	Value
Style	Dark academia — serious, elegant, minimal
Fonts	Primary: Inter / Geist. Mono: JetBrains Mono
Corners	Rounded (8–16px border-radius)
Shadows	Subtle elevation, no harsh drop shadows
Icons	Lucide React or Phosphor Icons
Illustrations	Line-art / flat medical illustrations
Animations	Subtle — fade, slide, scale. No heavy motion

6.2 Colour Palette

Token	Hex
--green-dark	#1A3C2E → Primary brand, backgrounds
--green-mid	#2D6A4F → Buttons, accents
--green-light	#74C69D → Highlights, success states
--beige-base	#F5F0E8 → Page backgrounds
--beige-light	#FAF7F2 → Card backgrounds
--blue-dark	#1A3550 → Headings, sidebar
--blue-accent	#2E86AB → Links, interactive
--blue-light	#D0E8F4 → Info callouts
--text-dark	#1C1C1C → Primary text
--text-mid	#555555 → Secondary text
--error	#C0392B → Errors, wrong answers
--gold	#B8860B → Blitz Mode accents

6.3 Component Library

► Core Components

- Button — Primary / Secondary / Ghost / Danger
- Card — with header, body, footer slots
- Badge — status, difficulty, mode indicators
- Timer — animated countdown ring (Blitz Mode)

- Progress Bar — linear and radial variants
- Modal — for room creation, confirmations
- Toast — success / error / info notifications
- Avatar — user display with fallback initials
- Leaderboard Row — rank, name, score, avatar
- Question Card — MCQ display with option grid
- Streak Counter — fire emoji with count
- Sidebar Nav — collapsible, icon + label

6.4 Page Layouts

Page	Layout Description
Home / Landing	Hero + feature sections + CTA — public
Dashboard	Left sidebar nav + main content area + stats widget
Lessons	Subject grid → unit list → module reader
MedQuest Hub	Mode selector cards + recent history
Free Mode	Question card centered, progress bar top
Blitz Mode	Full-screen timer ring + question + streak counter
Room Mode	Split: question left, player list + chat right
Statistics	Charts dashboard, filter controls top
Profile	Avatar + stats summary + friend list
Admin Panel	Table-based CRUD for content management

6.5 Responsiveness

- Desktop-first design (1280px+ primary target)
- Tablet layout collapses sidebar to bottom nav (768–1280px)
- Mobile layout stacks vertically, simplified navigation (<768px)
- Blitz Mode remains playable on mobile with adapted layout
- Room Mode mobile: tabs between question and chat

7. FRONTEND TECHNICAL SPECIFICATION (NEXT.JS)

7.1 Stack Overview

Tool / Library	Purpose
Next.js 14+ (App Router)	Framework — SSR, SSG, API Routes
TypeScript	Type safety across entire codebase
Tailwind CSS	Utility-first styling
shadcn/ui	Accessible component primitives
Zustand	Global state management
React Query (TanStack)	Server state, caching, background sync
Zod	Schema validation (forms + API responses)
React Hook Form	Form handling with Zod resolver
Socket.io-client	Real-time: Room Mode, chat, Blitz timer
Recharts	Statistics charts and graphs
Framer Motion	Subtle UI animations
next-auth / Supabase Auth	Authentication integration
date-fns	Date formatting and streak calculation
Lucide React	Icon library

7.2 Project Structure

Next.js App Directory Structure	
agora/	
├─ app/	
└─ (auth)/	
└─ login/ page.tsx	
└─ register/ page.tsx	
└─ reset-password/ page.tsx	
└─ (dashboard)/	
└─ layout.tsx (sidebar + auth guard)	
└─ dashboard/ page.tsx	
└─ lessons/	
└─ page.tsx (subject list)	
└─ [moduleId]/ page.tsx	

	└─ medquest/	
	└─ page.tsx	(mode selector)
	└─ free/	page.tsx
	└─ blitz/	page.tsx
	└─ room/	
	└─ page.tsx	(room list/create)
	└─ [roomId]/page.tsx	
	└─ statistics/	page.tsx
	└─ blog/	page.tsx
	└─ profile/	page.tsx
	└─ admin/	
	└─ layout.tsx	(admin guard)
	└─ lessons/	CRUD pages
	└─ questions/	CRUD pages
	└─ users/	management
	└─ api/	(Next.js route handlers)
	└─ layout.tsx	(root layout)
	└─ page.tsx	(landing page)
	└─ components/	
	└─ ui/	(shadcn primitives)
	└─ quiz/	(QuestionCard, Timer, etc.)
	└─ stats/	(Charts, StreakWidget)
	└─ room/	(RoomChat, PlayerList)
	└─ layout/	(Sidebar, Navbar)
	└─ lib/	
	└─ supabase.ts	(or django api client)
	└─ socket.ts	(socket.io setup)
	└─ store/	(Zustand stores)
	└─ validators/	(Zod schemas)
	└─ hooks/	(custom React hooks)
	└─ types/	(global TypeScript types)
	└─ public/	(static assets)

7.3 Key Frontend Modules

► Authentication Module

- Protected routes via Next.js middleware

- JWT stored in httpOnly cookies
- Auto-refresh logic with React Query
- Redirect to /login on 401 responses

► **Quiz Engine (Client-Side)**

- Manages question index, selected answer, score state
- Handles Blitz countdown via useInterval hook
- Dispatches socket events for Room Mode
- Saves session result on completion via API call

► **Real-Time Layer**

- Socket.io client connects on Room Mode entry
- Events: player-joined, answer-submitted, timer-update, game-end
- Optimistic UI — score updates immediately, reconciles with server
- Chat: message-send, message-receive, user-typing

8. BACKEND — OPTION A: SUPABASE

8.1 Why Supabase

Supabase is a Firebase alternative built on PostgreSQL. It provides authentication, database, storage, real-time subscriptions, and edge functions out of the box — ideal for rapid development with minimal backend infrastructure.

8.2 Supabase Services Used

Service	Usage in Agora
Supabase Auth	User sign-up, login, OAuth (Google), JWT sessions
PostgreSQL DB	All platform data — users, questions, sessions, rooms
Row Level Security	Per-user data access control without custom middleware
Realtime	Room Mode presence, live score updates, chat (Postgres changes)
Storage	User avatars, lesson images, admin-uploaded content
Edge Functions	Blitz timer logic, streak validation, leaderboard recalculation

8.3 Database Schema (Supabase / PostgreSQL)

► Table: users (extends auth.users)

Column	Type / Constraint
id	UUID PK — references auth.users(id)
username	TEXT UNIQUE NOT NULL
avatar_url	TEXT
year_of_study	SMALLINT
university	TEXT
role	TEXT DEFAULT 'student' CHECK (role IN ('student','admin'))
streak_current	INTEGER DEFAULT 0
streak_best	INTEGER DEFAULT 0
last_activity_date	DATE
is_premium	BOOLEAN DEFAULT false
created_at	TIMESTAMPTZ DEFAULT now()

► Table: subjects

Column	Type / Constraint
--------	-------------------

id	UUID PK DEFAULT gen_random_uuid()
name	TEXT NOT NULL
description	TEXT
icon_url	TEXT
display_order	SMALLINT DEFAULT 0
created_at	TIMESTAMPTZ DEFAULT now()

► **Table: lessons (modules)**

Column	Type / Constraint
id	UUID PK
subject_id	UUID FK → subjects(id) ON DELETE CASCADE
title	TEXT NOT NULL
content	TEXT (Markdown)
display_order	SMALLINT
is_published	BOOLEAN DEFAULT false
created_at	TIMESTAMPTZ
updated_at	TIMESTAMPTZ

► **Table: questions**

Column	Type / Constraint
id	UUID PK
lesson_id	UUID FK → lessons(id)
subject_id	UUID FK → subjects(id)
question_text	TEXT NOT NULL
options	JSONB — array of 4 strings
correct_index	SMALLINT (0–3)
explanation	TEXT
difficulty	TEXT CHECK (difficulty IN ('easy','medium','hard'))
created_at	TIMESTAMPTZ

► **Table: quiz_sessions**

Column	Type / Constraint
id	UUID PK

user_id	UUID FK → users(id)
mode	TEXT CHECK (mode IN ('free','blitz','room'))
lesson_id	UUID FK → lessons(id), nullable
subject_id	UUID FK → subjects(id), nullable
total_questions	INTEGER
correct_answers	INTEGER
duration_seconds	INTEGER
best_streak	INTEGER
completed_at	TIMESTAMPTZ DEFAULT now()

► **Table: question_answers (per-question log)**

Column	Type / Constraint
id	UUID PK
session_id	UUID FK → quiz_sessions(id)
question_id	UUID FK → questions(id)
selected_index	SMALLINT
is_correct	BOOLEAN
answered_at	TIMESTAMPTZ

► **Table: rooms**

Column	Type / Constraint
id	UUID PK
host_id	UUID FK → users(id)
lesson_id	UUID FK → lessons(id)
invite_code	TEXT UNIQUE (6-char alphanumeric)
status	TEXT CHECK (status IN ('waiting','active','finished'))
max_players	SMALLINT DEFAULT 10
question_count	SMALLINT DEFAULT 20
created_at	TIMESTAMPTZ
started_at	TIMESTAMPTZ
finished_at	TIMESTAMPTZ

► **Table: room_participants**

Column	Type / Constraint
id	UUID PK
room_id	UUID FK → rooms(id)
user_id	UUID FK → users(id)
score	INTEGER DEFAULT 0
joined_at	TIMESTAMPTZ
is_ready	BOOLEAN DEFAULT false

► Table: room_chat_messages

Column	Type / Constraint
id	UUID PK
room_id	UUID FK → rooms(id)
user_id	UUID FK → users(id)
content	TEXT NOT NULL (max 500 chars)
sent_at	TIMESTAMPTZ DEFAULT now()

► Table: friendships

Column	Type / Constraint
id	UUID PK
requester_id	UUID FK → users(id)
recipient_id	UUID FK → users(id)
status	TEXT CHECK (status IN ('pending', 'accepted', 'rejected', 'blocked'))
created_at	TIMESTAMPTZ
updated_at	TIMESTAMPTZ

■ 8.4 Row Level Security (RLS) Rules

- Users: read own row only. Admins read all.
- Lessons: all authenticated users can read published lessons.
- Questions: all authenticated users can read. Only admins write.
- quiz_sessions: user reads/inserts own rows only.
- Rooms: participants can read their room. Host can update.
- room_chat_messages: participants can read + insert in their room.
- friendships: users read/write rows where they are requester or recipient.

■ 8.5 Supabase Edge Functions

Function	Responsibility
update-streak	Called after session save — updates streak_current, streak_best, last_activity_date
generate-invite-code	Creates unique 6-char room code with collision check
calculate-blitz-score	Server-side validation of Blitz score to prevent cheating
update-leaderboard	Recalculates Blitz leaderboard rankings after each session
send-room-invite	Sends email/push notification to invited friend

■ 8.6 Real-Time (Supabase Channels)

Supabase Realtime uses Postgres changes and broadcast channels. For Room Mode:

- Channel: room:{roomId} — broadcasts player actions, scores, game state
- Postgres Changes on room_participants — detects join/leave
- Postgres Changes on room_chat_messages — delivers chat in real time
- Blitz Mode timer runs client-side, validated server-side on submit

9. BACKEND — OPTION B: DJANGO + DJANGO REST FRAMEWORK

9.1 Why Django

Django offers a battle-tested full backend with ORM, admin panel, auth, and REST framework — ideal for teams comfortable with Python and wanting maximum control over business logic. The real-time layer is handled separately via Django Channels (WebSocket).

9.2 Stack

Tool	Purpose
Django 5.x	Main framework — routing, ORM, admin
Django REST Framework	REST API — serializers, viewsets, authentication
Django Channels	WebSocket support — Room Mode, Blitz timer, chat
PostgreSQL	Primary database (same schema as Supabase option)
Redis	Channel layer for Django Channels + session/task caching
Celery + Redis	Async tasks: streak updates, email notifications
Simple JWT	JWT access + refresh token authentication
dj-rest-auth	Auth endpoints: login, register, password reset
django-cors-headers	CORS for Next.js frontend requests
Daphne / Uvicorn	ASGI server for WebSocket support
AWS S3 / Cloudflare R2	File storage (avatars, images)
Gunicorn + Nginx	Production HTTP server

9.3 Project Structure

Django Project Structure	
agora_backend/	
├─ config/	
│ └─ settings/	
│ │ └─ base.py	
│ │ └─ development.py	
│ │ └─ production.py	
│ └─ urls.py	
│ └─ asgi.py	(Channels support)
│ └─ wsgi.py	
└─ apps/	

	└─ users/	(auth, profile, friends, streak)
	└─ content/	(subjects, lessons, rich text)
	└─ quiz/	(questions, sessions, answers)
	└─ blitz/	(blitz mode, leaderboard)
	└─ rooms/	(room model, participant, chat)
	└─ stats/	(aggregated analytics views)
	└─ blog/	(posts, categories)
	└─ websockets/	
	└─ consumers/	
	└─ room_consumer.py	
	└─ blitz_consumer.py	
	└─ routing.py	
	└─ tasks/	(Celery tasks)
	└─ common/	(shared utils, permissions, mixins)
	└─ requirements/	
	└─ base.txt	
	└─ dev.txt	
	└─ prod.txt	

■ 9.4 Django Models Overview

► users app — UserProfile model

- OneToOne with Django's built-in User model
- Fields: username, avatar, year_of_study, university, role, streak_current, streak_best, last_activity_date, is_premium
- Custom manager for streak logic

► content app — Subject / Lesson models

- Subject: name, description, icon_url, display_order
- Lesson: ForeignKey(Subject), title, content (TextField/Markdown), display_order, is_published

► quiz app — Question / QuizSession / QuestionAnswer

- Question: FK(Lesson), FK(Subject), question_text, options (JSONField), correct_index, explanation, difficulty
- QuizSession: FK(User), mode, FK(Lesson/Subject nullable), scores, timestamps
- QuestionAnswer: FK(Session), FK(Question), selected_index, is_correct

► rooms app — Room / RoomParticipant / RoomChatMessage

- Room: FK(host), FK(Lesson), invite_code, status, max_players, question_count, timestamps
- RoomParticipant: FK(Room), FK(User), score, is_ready, joined_at

- RoomChatMessage: FK(Room), FK(User), content, sent_at

9.5 REST API Endpoints

Auth Endpoints

Method + Path	Description
POST /api/auth/register/	Create account
POST /api/auth/login/	Login, returns JWT pair
POST /api/auth/token/refresh/	Refresh access token
POST /api/auth/password/reset/	Send reset email
POST /api/auth/logout/	Blacklist refresh token

User / Profile Endpoints

Method + Path	Description
GET /api/users/me/	Get own profile
PATCH /api/users/me/	Update profile
GET /api/users/{id}/	Public profile view
GET /api/users/search/?q=name	Search users by username
GET /api/users/me/friends/	List accepted friends
POST /api/users/me/friends/request/	Send friend request
PATCH /api/users/me/friends/{id}/	Accept / reject request
DELETE /api/users/me/friends/{id}/	Remove friend

Content Endpoints

Method + Path	Description
GET /api/subjects/	List all subjects
GET /api/subjects/{id}/lessons/	Lessons for a subject
GET /api/lessons/{id}/	Single lesson detail
POST /api/lessons/	Create lesson (admin only)
PUT /api/lessons/{id}/	Update lesson (admin only)
DELETE /api/lessons/{id}/	Delete lesson (admin only)

Quiz Endpoints

Method + Path	Description
GET /api/questions/?lesson_id=X	Get questions for lesson
POST /api/sessions/	Save completed session
GET /api/sessions/	List user sessions (paginated)
GET /api/sessions/{id}/	Single session detail

► Room Endpoints

Method + Path	Description
POST /api/rooms/	Create room
GET /api/rooms/{code}/	Get room by invite code
POST /api/rooms/{id}/join/	Join room
POST /api/rooms/{id}/start/	Start game (host only)
GET /api/rooms/{id}/results/	Final results

■ 9.6 WebSocket Consumers (Django Channels)

► RoomConsumer — ws://api/ws/room/{roomId}/

Event (client → server)	Action
player_ready	Mark participant as ready
submit_answer	Record answer, update score
send_message	Broadcast chat message to room
leave_room	Remove participant from room
Event (server → client)	Action
player_joined	New participant data broadcast
score_update	Updated scores for all players
new_message	New chat message delivered
game_started	Question set + timer delivered to all
game_ended	Final leaderboard broadcast

■ 9.7 Celery Async Tasks

- `update_streak(user_id)` — runs after each session save
- `send_friend_request_email(user_id, friend_id)` — notification email
- `send_room_invite_notification(user_id, room_code)` — push/email
- `recalculate_blitz_leaderboard(lesson_id)` — triggered after Blitz session
- `send_weekly_summary_emails()` — scheduled weekly via Celery Beat

10. SUPABASE VS DJANGO — DECISION GUIDE

10.1 Feature Comparison

Feature	Supabase
Setup speed	Very fast — managed platform
Auth	Built-in (OAuth, JWT, magic link)
Database	PostgreSQL (hosted)
Real-time	Built-in (Postgres changes + broadcast)
Storage	Built-in (S3-compatible)
Custom business logic	Edge Functions (Deno/TypeScript)
Admin panel	Table Editor UI (basic)
Pricing	Free tier, then usage-based
Maintenance	Managed — minimal DevOps
Learning curve	Low
Team language	TypeScript / SQL
Scalability	High (Supabase manages infra)

Feature	Django
Setup speed	Moderate — requires server setup
Auth	Built-in + dj-rest-auth + JWT
Database	PostgreSQL (self-hosted or managed)
Real-time	Django Channels + Redis (more setup)
Storage	S3/R2 via django-storages
Custom business logic	Full Python — unlimited flexibility
Admin panel	Powerful Django Admin built-in
Pricing	Server cost (VPS: ~\$5-20/mo)
Maintenance	You manage server, updates, scaling
Learning curve	Medium-High
Team language	Python
Scalability	Horizontal scaling possible but manual

10.2 Recommendation

Our Recommendation

START with Supabase for MVP and early growth.

It eliminates infrastructure overhead, provides real-time out of the box,
and lets the team focus entirely on product and user experience.

MIGRATE to Django (or hybrid) when:

- Complex business logic exceeds what Edge Functions handle cleanly
- Custom admin tooling is required beyond the Supabase editor
- The team has Python expertise and needs tighter backend control
- Monthly Supabase costs exceed a self-hosted server's cost

11. DEVELOPMENT ROADMAP & PHASES

11.1 Overview

Agora is developed in 5 phases within a total estimated timeline of 2.5 months (10 weeks). Each phase has a clear goal, scope, and acceptance criteria. The timeline is aggressive and requires disciplined sprints with no scope creep.

Timeline Summary

Total Duration : 2.5 months (10 weeks)		
Phase 0	: Week 1	— Setup & tooling
Phase 1	: Weeks 2-3	— Auth + Lessons
Phase 2	: Weeks 4-5	— MedQuest Free & Blitz
Phase 3	: Weeks 6-7	— Room Mode + Friends
Phase 4	: Week 8	— Statistics + Blog
Phase 5	: Weeks 9-10	— Testing, polish & launch

Phase 0 <i>Week 1 (Days 1-7)</i>	Goals
	✓ Project setup, tooling, monorepo structure
	✓ Next.js project initialised with TypeScript + Tailwind
	✓ Supabase project created, tables seeded with schema
	✓ Authentication flow working (email/password)
	✓ Git repository, branching strategy, CI pipeline
	✓ Design system tokens defined in Tailwind config

Phase 1 <i>Weeks 2-3 (Days 8-21)</i>	Goals
	✓ User authentication complete (register, login, logout, reset password)
	✓ User profile page (view + edit)
	✓ Lessons section: subject list → unit list → module reader
	✓ Admin panel: create / edit / delete lessons
	✓ Basic navigation and layout with sidebar
Phase 2 <i>Weeks 4-5 (Days 22-35)</i>	✓ Mobile-responsive base layout

Phase 2 <i>Weeks 4-5 (Days 22-35)</i>	Goals
	✓ MedQuest Free Mode — full quiz flow with session saving
	✓ MedQuest Blitz Mode — countdown timer, +5/-5 seconds, streak
	✓ Question management in admin panel
	✓ Quiz session history saved and displayed
	✓ Basic Statistics page (accuracy, sessions played)
	✓ Streak system initialised

Phase 3 <i>Weeks 6–7 (Days 36–49)</i>	Goals
	✓ Room Mode — create room, invite code, real-time multiplayer
	✓ In-room chat system
	✓ Real-time score leaderboard during game
	✓ End-of-game results screen
	✓ Friend system — add, accept, remove friends
	✓ Room invite via friend list

Phase 4 <i>Week 8 (Days 50–57)</i>	Goals
	✓ Full Statistics dashboard with charts (Recharts)
	✓ Performance breakdown by subject and lesson
	✓ Blitz Mode global + friends leaderboard
	✓ Daily streak notifications
	✓ Blog section — admin posts, public reader view
	✓ SEO optimisation for landing page

Phase 5 <i>Weeks 9–10 (Days 58–70 — End of Month 2.5)</i>	Goals
	✓ End-to-end testing (Playwright)
	✓ Performance audits (Lighthouse 90+ scores)
	✓ Security audit (SQL injection, XSS, CSRF, RLS review)
	✓ Staging environment + production deployment
	✓ User beta testing with real students
	✓ Bug fixes and polish round
	✓ Launch 🚀

11.2 Sprint Structure

Sprint Length	1 week (compressed — 2.5 month total budget)
Sprint Planning	Every Monday — define tasks for the week
Daily Standup	15 min async (Notion/Slack update)
Sprint Review	Friday — demo, review completed tasks
Retrospective	Monthly — process improvements
Issue Tracker	GitHub Issues + Milestones
Branch Strategy	main → staging → feature/* branches
PR Reviews	Required for all merges to main

12. MVP SCOPE & LAUNCH CRITERIA

12.1 In Scope (MVP)

- User registration, login, profile
- Lessons section (view only — at least 2 subjects seeded)
- MedQuest Free Mode
- MedQuest Blitz Mode
- MedQuest Room Mode (real-time)
- Friend system (add / remove / invite to room)
- Basic statistics page
- Daily streak tracking
- Admin panel (lessons + questions management)

12.2 Out of Scope (MVP — Post-launch)

- Blog section (visible but empty)
- Premium subscription system
- Mobile apps (iOS / Android)
- Achievement badges
- XP / levelling system
- Advanced statistics with AI insights
- Email notifications (streak reminders)
- Public leaderboard pages

12.3 Launch Acceptance Criteria

Criterion	Requirement
Authentication	100% of auth flows work without errors
Lessons	At least 3 subjects, 10 lessons, 100 questions loaded
Free Mode	Complete quiz with score and session save
Blitz Mode	Timer works, scores validated server-side
Room Mode	2+ simultaneous users complete a room game
Performance	Page load < 2s on desktop, < 3.5s on mobile (LCP)
Mobile	All MVP pages functional on 375px width
Security	RLS tested, no exposed private data
Uptime	99% uptime in staging for 1 week before launch

13. SECURITY SPECIFICATION

■ 13.1 Authentication Security

- Passwords hashed with bcrypt (min cost factor 12) or Argon2id
- JWT access tokens: 15-minute expiry
- Refresh tokens: 7-day expiry, stored in httpOnly cookies
- CSRF protection on all state-changing endpoints
- Rate limiting on login: max 5 attempts per 15 minutes per IP
- OAuth via Supabase Auth or django-allauth — no custom OAuth logic

■ 13.2 Data Protection

- All API endpoints require authentication (except public landing page)
- RLS on all Supabase tables — users cannot access others' data
- Django: IsAuthenticated permission class on all views
- Admin routes protected by role check (role === 'admin')
- Input validation with Zod (frontend) and DRF serializers (backend)
- Parameterised queries everywhere — no raw string SQL

■ 13.3 Real-Time Security

- WebSocket connections authenticated via JWT before connection accepted
- Room actions validated server-side — client score not trusted
- Chat message length capped at 500 characters server-side
- Rate limiting on chat: max 1 message per second per user

■ 13.4 Infrastructure

- HTTPS everywhere — TLS 1.3 required
- Environment variables for all secrets — never committed to git
- Content Security Policy headers configured
- CORS: only allow frontend domain, no wildcard origins
- Regular automated dependency audits (npm audit, pip-audit)

14. TESTING STRATEGY

14.1 Testing Layers

Layer	Tool
Unit tests — Frontend components	Vitest + React Testing Library
Unit tests — Backend models/services	pytest (Django) / Supabase local
Integration tests — API endpoints	pytest + DRF test client / Supertest
End-to-End tests — full user flows	Playwright
Performance	Lighthouse CI, k6 (load testing)
Accessibility	axe-core + manual keyboard testing

14.2 Critical Test Cases

► Authentication

- User can register with valid data
- Duplicate email is rejected
- Login with wrong password is rejected
- JWT token refresh works

► Quiz Modes

- Free Mode: all questions answered → session saved with correct score
- Blitz Mode: timer runs down, correct adds 5s, wrong removes 5s
- Blitz Mode: timer reaching 0 ends session, score validated server-side
- Room Mode: 2 users join, complete quiz, receive correct leaderboard

► Real-Time

- User A submitting answer is reflected on User B screen within 500ms
- Chat message delivered to all room participants within 300ms
- Player disconnect removes them from participant list

► Access Control

- Unauthenticated user cannot access /dashboard
- Non-admin user cannot create/edit lessons via API
- User cannot read another user's quiz sessions

15. DEPLOYMENT & INFRASTRUCTURE

15.1 Environments

Environment	Purpose
Local	Developer machine — full stack with local DB or Supabase dev project
Staging	Production mirror — auto-deploys from main branch
Production	Live platform — manual deploy trigger or tag-based

15.2 Frontend Deployment (Next.js)

- Platform: Vercel (recommended) or AWS Amplify
- Auto-deploy from GitHub on push to main
- Preview deployments for every PR
- Environment variables managed in Vercel Dashboard
- Edge network CDN for static assets
- ISR (Incremental Static Regeneration) for lesson pages

15.3 Backend Deployment

► Supabase Option

- Supabase hosts database, auth, storage, edge functions
- No separate server needed for MVP
- Migrations managed via Supabase CLI (supabase db push)
- Edge Functions deployed via supabase functions deploy

► Django Option

- Platform: Railway, Render, or AWS EC2
- Gunicorn + Nginx for HTTP
- Daphne for ASGI/WebSocket
- PostgreSQL: managed Neon, AWS RDS, or Supabase Postgres
- Redis: Upstash Redis or AWS ElastiCache
- Static files: served via Whitenoise or S3/CloudFront
- CI/CD: GitHub Actions → build → test → deploy

15.4 CI/CD Pipeline

GitHub Actions Pipeline
Trigger: Push to main or pull request
Step 1: Lint — ESLint + Prettier (frontend), Ruff/flake8 (backend)

Step 2: Type Check — tsc --noEmit
Step 3: Unit Tests — Vitest / pytest
Step 4: Build — next build (check for errors)
Step 5: E2E Tests — Playwright (on staging)
Step 6: Deploy — Vercel CLI / Railway CLI
Step 7: Notify — Slack/Discord on success or failure

16. MONITORING & OBSERVABILITY

16.1 Tools

Tool	Purpose
Vercel Analytics	Web vitals, page views, performance
Sentry	Error tracking — frontend & backend exceptions
Uptime Robot / BetterUptime	Uptime monitoring + alerts
Supabase Dashboard	DB performance, query logs, realtime stats
PostHog (optional)	Product analytics — user flows, feature flags
Datadog / Grafana (Django)	Infrastructure metrics, API latency

16.2 Key Metrics to Track

- Daily Active Users (DAU) and Weekly Active Users (WAU)
- Session completion rate per quiz mode
- Average session duration (engagement metric)
- Streak retention (% users maintaining 7-day streaks)
- Room Mode completion rate (rooms started vs finished)
- API p95 latency (target: < 200ms)
- Error rate (target: < 0.1%)
- New user registration funnel conversion

17. BUSINESS MODEL & MONETISATION

17.1 Phase 1 — Free Launch

The platform launches completely free. The objective is user acquisition, retention data, and community building among medical students.

17.2 Phase 2 — Premium Subscription (post-MVP, priced in DZD)

Subscription pricing is set entirely in Algerian Dinar (DZD) to ensure accessibility for Algerian medical students without reliance on foreign payment infrastructure.

Premium Feature	Description
Advanced Statistics	Full performance analytics, weakness reports, AI insights
Streak Protection	Freeze up to 3 days per month — streak preserved
Exclusive Content	Premium lesson content, exam-focused question packs
Unlimited Blitz	No daily Blitz limit (free users: 10 sessions/day)
Priority Matching	Faster room matchmaking
Ad-Free Experience	Remove any future ads from free tier
Download for Offline	PDF export of lessons (future)

17.3 Pricing (Indicative — Algerian Dinar, DZD)

- Monthly plan: 500–1 000 DZD / month (~3.5–7 USD equivalent)
- Annual plan: 4 000–8 000 DZD / year — ~35 % saving vs monthly
- Student discount tier: 300 DZD / month (with university email verification)
- Group / University plan: negotiated licensing — faculties & prépa schools (future B2B)
- Payment methods: CIB card, Edahabia (Algérie Poste), future BaridiMob integration
- Note: All prices set in DZD — no foreign currency dependency for Algerian users

18. GLOSSARY & CODING CONVENTIONS

18.1 Glossary

Term	Definition
QCM	Question à Choix Multiple — Multiple Choice Question
Blitz Mode	Timed speed-quiz with dynamic countdown timer
Room Mode	Real-time multiplayer quiz room
Streak	Consecutive days with at least one activity
Session	A single completed quiz attempt (any mode)
Module	An individual lesson within a unit
Unit	A grouping of modules within a subject
RLS	Row Level Security — PostgreSQL per-row access control
JWT	JSON Web Token — stateless auth credential
Edge Function	Serverless function running at CDN edge (Supabase/Vercel)

18.2 Coding Conventions

► Frontend (TypeScript/React)

- Components: PascalCase (QuestionCard.tsx)
- Hooks: camelCase prefixed with 'use' (useBlitzTimer.ts)
- Constants: UPPER_SNAKE_CASE
- API calls: grouped in lib/api/ — one file per domain
- Types: defined in types/ or co-located with component
- No 'any' — use unknown and narrow types

► Backend (Django — if chosen)

- Models: PascalCase (QuizSession)
- Views: PascalCase + 'View' suffix (QuizSessionListView)
- Serializers: PascalCase + 'Serializer' suffix
- URL names: snake_case (quiz-session-list)
- All API endpoints versioned: /api/v1/...
- Docstrings on all model methods and view classes

18.3 Git Conventions

Type	Example commit message
feat	feat: add Blitz Mode timer component

fix	fix: correct streak reset logic on UTC midnight
chore	chore: update dependencies
docs	docs: add API endpoint documentation
test	test: add Playwright test for Room Mode flow
refactor	refactor: extract quiz engine into custom hook
style	style: apply Tailwind dark mode classes to sidebar

19. VISION & LONG-TERM ROADMAP

19.1 Platform Vision

"Agora aims to become the definitive learning companion for every Algerian medical student — a complete ecosystem where knowledge meets competition, structured study meets social engagement, and every session brings you closer to mastery. Built in Algeria, priced in DZD, designed for the realities of medical studies in Algeria."

19.2 Long-Term Feature Roadmap

Horizon	Features
6 months	Premium subscription launch, achievement badges, email notifications
12 months	Mobile apps (React Native), AI-powered question recommendations
18 months	University partnerships, B2B licensing to schools
24 months	Live video study rooms, mentor system, peer tutoring
36 months	Expansion to pharmacy, dentistry, science degrees
48 months+	Internationalisation (EN, AR), global medical community

19.3 Success Metrics

- 500 DAU within 3 months of launch (target audience: Algerian medical students)
- 10,000 registered users within 6 months
- 30% 30-day retention rate
- Average 3+ sessions per user per week
- 5% conversion rate to premium within 12 months of paid launch

END OF SPECIFICATION DOCUMENT
Agora Platform • Version 1.0 • Confidential